```
~> echo "$RANDOM$RANDOM$RANDOM$RANDOM $RANDOM$RANDOM$RAND
OM$RANDOM" | perl -lane '$k = rand();
    if ($k < .4) { print "$F[0] * $F[1]"; next }
    elsif ($k < .8) { print "$F[0] / $F[1]"; next }
    elsif ($k < .9) { print "$F[0] + $F[1]"; next }
    else { print "$F[0] - $F[1]" }' | print
21425451339782729 24 * 16467240779233 1690
~>
```

The next run gave us:

```
83232139721890220 85 - 29706112412437431635

So far so good—I was getting on...
```

Getting a data hose

Z.sh/Bash have in-built *for* loops. So we converted the left hand trickle into a hose...

```
~> for ((i = 0; i < 5; ++i))
do
 echo "$RANDOM$RANDOM$RANDOM$RANDOM $RANDOM$RANDOM$RAND
OM$RANDOM"
done |  perl -lane '$k = rand();
pipe quote>     if ($k < .4) { print "$F[0] * $F[1]"; next }
pipe quote>     elsif ($k < .8) { print "$F[0] / $F[1]"; next }
pipe quote>     elsif ($k < .9) { print "$F[0] + $F[1]"; next }
pipe quote>     else { print "$F[0] - $F[1]" }' | tee > expressions.txt | bc > results.txt
203431407232304631628 / 12805234313157116052
185372667129981605 1 + 29694216342188821346
293931704299361562 5 / 38033016128821552
113881012327974203 + 18803135732414463 78
276452368052387683 * 14343404223735113
~>
```

Now, we just increased the *for* loop runs**,** dumped the stuff into a file, and we had our test cases. 50000 such computations would be fine for us.

Computing the expected answers was simple—bc is always around, ready to do our bidding.

With a tee, I captured the test cases to the file, ***expressions.txt***. The expected results went into ***results.txt***.

A dollop of Ruby

However, I skirted the real issue: how were we to get random numbers that were around 24 digits—that is where our boundary conditions had to be validated...

```
~> ruby -e '100.times { print "#{rand(10000000000000000000000)}\n" }'
```

And no, don't try and tap out those zeroes. After you enter 1, press Alt 23, release Alt, and hit 0 – then the command line taps out 23 zeroes for you—pretty cool, right?

Just to make sure we get many numbers with 23 digits in them, you can quickly run a test...

```
~> ruby -e '100.times { print "#{rand(10000000000000000000000)}\n" }' | awk
'length < 23' | wc -l
6
~>
```

I played with this by changing the loop counter 100 to a few other numbers and found that we got the desired numbers 90 per cent of the time—and that's good.

Could we just step further and generate the pair of them? Oh yes, with some more Rubyism.

```
~> ruby -e 'def x
quote>          rand(10000000000000000000000)
quote>      end
quote>      100.times { m,n = x, x
quote>      print "#{m} #{n}\n" }'
183145950167277604111398 840402366385781446620 58
395443216053217248850550 27777137155951945530143
```

We could take out the shell loop above and instead use the above Ruby code in its place.

TIMTOWTDI (Perl speak)

Well yes, using Linux's */dev/urandom* bucket you can pick as many as you want:

```
od -An -tu /dev/urandom | awk -vOFS="" '{$1=$1; print}' | sed -n
's/\(.\{24\}\).*/\1/;p' | sed -n 'h; s/\n//;p'*
```

There are a couple of awk and sed idioms here, however it is a nice exercise to open the info pages and figure out how these work. This, by the way, makes sure that we get each number with 24 digits in it.

We can make absolutely sure of this, but I will leave it as an exercise.

Voila!

And the test data is ready.

The rest was all easy and routine—we pulled these files into a CppUnit test method and called tommath APIs with the expression and compared the result.

And we found the answer in an hour—tommath faithfully produced the correct output for numbers with 27 digits in them, which was enough for us to choose it...

Problem solved. Open Source is totally awesome! END

By: Atul Shriniwas Khot

The author works at IBM Software Labs in Hyderabad as a senior software engineer, and has been dabbling with UNIX/Linux for the last 14 years. He is into Java/J2EE, Groovy and Ruby these days, but at times he hankers after dear-old C++ and Perl. He loves design patterns, algorithms, multi-threading and refactoring code to make it stylish. And of course, he loves vim (never miss a chance to use it) and zsh. He collects classic British mysteries (especially those green and white Penguin crime classics—penguins make life so delectable;-))